PR Code Review Checklist (for Devs)

⚠ Ensure you review the following list of items before raising a PR to avoid rejections and rework. It's important that every developer understands what's expected like coding standards, general method naming conventions, security vulnerabilities, etc.

1. Don't COPY and PASTE code from legacy codebases!
   - Your job is to upgrade it to new standards without losing any semantic meaning.
   - If you have copied code, make necessary code changes before raising a PR!
   - Ideally, we want a `Controller` → `Service` → `Repository` based implementation.
     - A `Controller` can only talk to a `Service` or multiple `Services`.
     - A `Service` can talk to its own `Repository` or call other `Services`. Avoid dependency on multiple repositories within a single service since validation and other business rules shouldn't be duplicated. In addition to this, services can throw exceptions for invalid situations which can leveraged directly in other services.
     - A `Repository` is a simple data store for retrieving, saving and deleting entities with no business logic or complicated data management mechanisms. It may depend on other repositories in some cases to avoid duplications.
2. Avoid using `null` and unnecessary null checks throughout the codebase.
   - Only exception to this rule is on the repository layer, when a method returns a single value which may or may not be found. In such cases, the method signature should clearly indicate null return. In case of multiple values return an empty collection, ALWAYS!.
     - Eg: `Task<Customer?> FindCustomerAsync(long customerId, CancellationToken cancel);`
     - NOTE: how the return value says `Customer?` (nullable) and NOT `Customer`
     - `Repository` will NEVER THROW AN EXCEPTION!
     - `Services` and `Controllers` can NEVER return a `null` under any circumstances. Instead, they will throw customised & meaningful exceptions.
3. `Controllers, Services and Repositories` are 3 tiers which communicate via Domain objects only and not DTO or entities.
   - So, a repository layer will return a fully constructed instance of a Domain object if it exists in the database.
   - Services and Controllers will work with these objects and should return them back to the client. In some cases, all properties of a domain object may not be required in a response, for that use anonymous classes with a subset of those properties for projections.
     - For example: if you need a customer's first & last names only in a response, we could achieve this by:

```
var customers = await customerService.GetCustomersAsync(skip, count, cancel);
var firstAndLastNames = customers.Select(c => new {
  c.FirstName,
  c.LastName
});
```

4. Service classes should inherit from `ServiceBase` and should implement `IService`
5. Use meaningful variable/method names.
   - Variable names must be descriptive like `dataReader` and not `rdr`!!
   - Use `var` keyword wherever possible or `TypeName variableName = new { }` syntax but not both together!
   - Always use collection initialisation expression: `List<T> items = [];` to promote better code readability.
6. Asynchronous methods should have an `Async` suffix.
   - Methods returning multiple values should return an `IEnumerable<T>` where `T` is the data type.
   - Don't forget to pass a cancellation token to Async methods.
     - Example: `Task<IEnumerable<Customer>> GetCommunicationPreferencesAsync(long customerId, CancellationToken cancel);` And avoid making them optional with default values.
     - Interfaces and Repositories should always return `IEnumerable<T>` for Get or read-only methods to avoid accidental modifications.
7. Use file-scoped `namespaces` to reduce indentation.
8. Always use function scoped `using` statements and avoid unnecessary `try...catch` blocks to improve code readability.
9. Remove extra lines and unused `imports`.

10. Ensure there are no typos in method or variable names.
11. Always initialise model properties with default values that aren't nullable.
12. Don't create Helper methods as they are unnecessary.
13. Be judicious when creating extension methods and needs to be discussed with leads prior.
14. Implement proper exception handling (don't catch any Generic `Exception`)
15. Validate all domain objects and only domain object should contain validation logic.
    - Use .NET 8 `Attribute` based annotations wherever possible.
    - If this won't suffice, only then implement `IValidatableObject` interface.
    - If a validation requirement transcends this, only then implement Business or Inference rules.
    - DON'T add validation methods into services or interfaces unless there is a specific use-case where validation is provided as a service to other services.
16. Add proper XML documentation.
    - Ensure you have populated empty comments.
    - Don't raise a PR with empty comments, use Co-pilot to generate documentation and verify it afterwards.
    - Use meaningful comments that truly match what methods are actually doing.
17. Get an internal peer review with your respective offshore lead if required.
    - [MANDATORY] Attach `SonarQube` & `CheckmarX` reports with passed status (no medium or high issues).
      i. 5. **SonarQube Quality Gate Criteria**
      The following thresholds must be met unless explicitly exempted:
      - **Security Issues**: 0
      - **Reliability Issues**: 0
      - **Maintainability Issues**: 0
      - **Security Hotspots**: 0
      - **Code Coverage**: ≥ 90%
      - **Code Duplication**: < 2%

      6. **Checkmarx Security Scan Criteria**
      - **SAST Vulnerabilities**: 0
      - **SCA Vulnerabilities**: 0
      - **API Vulnerabilities**: 0
      - **IaC Vulnerabilities**: 0
      - **Container Vulnerabilities**: 0

      7. **Exemptions**
      - Any deviation from the above standards (e.g., failing quality gates or unresolved findings) must be formally approved by the **Technology Manager** or the **Head of Technology**.
    - [MANDATORY] Attach Postman or Swagger screenshots showing successful local result.
    - [MANDATORY] Code Coverage should be >= 90% as a bare minimum.
    - [MANDATORY] `Sample Request` & `Sample Response` sections must be populated with appropriate data and to the right standard, for every endpoint on its respective confluence page, see example below:
      i. You should place appropriate headers and other specific request related information in the `"Sample Request"` section to explain endpoint requirements such as pre-requisites, dependencies, specific mandatory headers, etc...
      ii. In cases where sample requests aren't applicable, populate these sections with `"NA"` whenever appropriate.
      iii. PR's will **NOT** be approved unless this section is filled out with correct information to the right standard! i.e. to a point where a tester can read and independently test that endpoint and complete their verification.
18. Ensure you add regions for List, Create, Update and Delete methods in your unit test class files ONLY (not in services or interfaces or any other places) so that testing team can verify your test cases easily. Also ensure proper naming convention is followed for Unit/Integration test file names → For example: `MedicareCardUnitTests.cs` or `MedicareCardIntegrationTests.cs`
    - Liaise with other team members if needed and rebase to improve code coverage.
19. Ensure you following all the above guidelines before raising a PR, your code is your responsibility.
20. Review this document periodically for updates.